**你說：**

幫我根據 tensorrt-llm 的職缺可能會問的幫我爬一下可能會問的問題, 我想要準備, 幫我查一下需要甚麼東西補一下, 像是
---
期待多輪面試，包括側重於 CUDA 核心優化、記憶體合併和共享記憶體使用模式的編碼挑戰。他們可能會給你一個要優化的核心，並要求你逐步講解你的思考過程。系統設計的面試往往圍繞分散式訓練架構、模型服務管道以及如何處理大模型的記憶體限制。對於記憶體層級的深入了解非常關鍵——你需要了解 L1/L2 快取行為、全局記憶體訪問模式，以及不同記憶體類型如何影響佔用率。
在技術深入探討方面，他們喜歡問有關 Tensor Core 利用率、混合精度訓練的影響，以及如何在實際部署情境中調試性能瓶頸的問題。你的 TensorRT 和 Triton 經驗肯定會被提起，因此準備好討論你所用的具體優化技術及其權衡。在算法方面，預期會有關於變壓器架構、注意力機制以及不同模型架構如何對應 GPU 計算模式的問題。對於 DSA，專注於平行算法、歸約操作，以及任何與樹結構相關的內容，因為這些內容經常出現在 GPU 算法設計中。面試官們真的很聰明，會注意到任何拐彎抹角的說法，所以確保你能解釋每個優化選擇背後的"為什麼"。

---

之類的, 我再慢慢準備
…
顯示較多

處理時間為 4m 29s

**ChatGPT 說：**

## NVIDIA TensorRT-LLM 面試準備 Roadmap

我查到一個非常直接對應你目標的 NVIDIA 台灣職缺：

**AI Computing Software Development Engineer, TensorRT-LLM** ↗ 官方職缺

NVIDIA · 台北 · Full-time · JR2022327

這個職缺實際要求的工作包括：

- LLM inference 的 performance analysis、optimization。
- 實作 GPU kernels、runtime features，支援新模型及推論演算法。
- 維護能跨平台擴充的 inference software。
- 熟悉 Python、PyTorch、Hugging Face、LLM architecture。
- 加分項目包含 C/C++、CUDA、profiling、inference framework（TensorRT-LLM、vLLM、SGLang）、compiler。

官方標示 3 年以上相關開發經驗；這不是專門為新鮮人開的職缺，但很適合作為準備技術深度的標準。

NVIDIA Cor… +1

### 1. 先釐清面試最可能考什麼

根據職缺、官方 TensorRT-LLM 技術內容，以及找到的公開面試經驗，我會把準備重心排成這樣：

| 面試領域 | 優先級 | 需要達到的能力 |
|---|---|---|
| CUDA Programming / GPU Architecture | P0 | 手寫 kernel、分析 memory access、優化效能 |
| LLM Inference Architecture | P0 | 解釋 prefill、decode、attention、KV cache |
| Performance Profiling | P0 | 用 Nsight 找 bottleneck，提出並驗證優化 |
| C++ / Python Coding & DSA | P0 | 正確、清楚地完成 coding challenge |

| | | |
|---|---|---|
| TensorRT-LLM Runtime / Serving | P0 | 理解 scheduling、batching、memory management |
| Tensor Core / GEMM / Triton | P1 | 能分析 tiling、fusion、precision 的 trade-off |
| Quantization | P1 | 理解 INT4、INT8、FP8、FP4 的實際影響 |
| Multi-GPU / Distributed Inference | P1 | TP、PP、EP、NCCL、communication overhead |
| Distributed Training | P2 | 理解基本策略，不必一開始深入訓練框架 |

這是針對 **TensorRT-LLM inference software engineering** 的排序，不代表 NVIDIA 官方公布了固定的面試題型。

你貼的描述大方向合理，但我會做兩個調整：

1. **Inference 比 distributed training 更值得先準備。** 例如 KV cache allocation、continuous batching、TTFT/TPOT，這些更直接對應目標團隊。
2. **不要只準備 CUDA 理論。** 面試官可能問你怎麼證明 kernel 真的更快、benchmark 是否公平，以及它是否改善 end-to-end latency。

**公開找到的真實面試經驗**

一名應徵 NVIDIA **Deep Learning Software Engineer – Inference** 的候選人，於 2025 年 6 月分享第一輪 technical screen：

- 深入追問 CUDA 經驗、實作過哪些 kernel、採取哪些優化，以及遇到什麼困難。
- 追問 vLLM、model serving 和個人實際貢獻。
- DSA 題目是 **Serialize and Deserialize Binary Tree**，重視乾淨的程式碼與時間、空間複雜度。

這是單一候選人的經驗，不是 TensorRT-LLM 團隊的官方題庫，但足以說明 **project deep dive + coding + inference systems** 都要準備。

LeetCode +1

## 2. 面試問題庫：先掌握這 32 題

以下除了特別標示的候選人回報題目以外，都是我依據職缺與實際工作內容整理的**模擬面試題**。

### A. CUDA / GPU Architecture（最高優先）

這部分我認為最值得花時間，因為它不只考你知不知道 CUDA API，而是會測試你能不能從硬體角度分析效能。

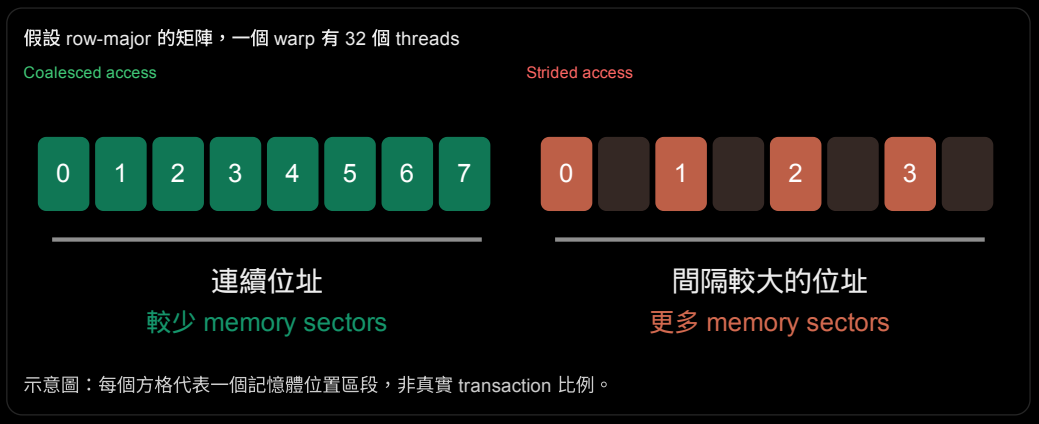
| 可能的面試題目 | 至少要回答到的重點 |
|---|---|
| 1. Explain memory coalescing. | Warp 的 threads 如何合併 global memory requests、stride 和 alignment 的影響 |
| 2. Global / shared / register / local memory 差異？ | 存放位置、存取行為、容量限制、register spilling、occupancy |
| 3. What are shared memory bank conflicts? | Bank mapping、32-way conflict、broadcast 特例、padding 技巧 |
| 4. How do you implement parallel reduction? | Tree reduction、warp shuffle、跨 block reduction、同步與數值誤差 |
| 5. Optimize a naive matrix transpose kernel. | Coalesced load/store、shared memory tiling、`32×33` padding |
| 6. Why isn't higher occupancy always better? | Register pressure、shared memory、memory latency hiding、instruction-level parallelism |
| 7. How would you implement tiled GEMM? | Global → shared → register 的資料重用、tile sizes、Tensor Cores |
| 8. What causes warp divergence? | SIMT execution、branch reconvergence、predication、資料分佈 |

**第 5 題是非常好的實作練習。**

NVIDIA 官方有一篇完整的 CUDA transpose 優化案例，從 naive transpose 到 shared-memory tiled transpose，再到解決 bank conflict。

NVIDIA Tec… +1

例如：



示意圖：每個方格代表一個記憶體位置區段，非真實 transaction 比例。

面試官可能繼續問：

> 既然 shared memory 比 global memory 快，是不是所有 kernel 都應該使用 shared memory？

正確思考方向是：不一定。Shared memory 的好處通常來自 **資料重用、coalescing 或跨 thread 的資料交換**。如果只是把資料多複製一次，反而增加指令、同步成本和 shared memory 使用量，可能比直接讀 global memory 更慢。

**B. LLM Inference / Transformer**

| 可能的面試題目 | 要回答到的重點 |
|---|---|
| 9. Why is prefill different from decode? | Prefill 的矩陣運算、decode 的 GEMV/small GEMM 與 KV cache bandwidth |
| 10. Derive KV cache memory usage. | Layers、KV heads、head dim、sequence length、batch、dtype |
| 11. MHA vs MQA vs GQA? | KV head 數量、共享方式、memory bandwidth 和模型品質 |
| 12. How does FlashAttention work? | IO-aware tiling、online softmax、避免 materialize $N \times N$ attention matrix |
| 13. What is PagedAttention? | Fixed-size blocks、logical-to-physical mapping、fragmentation、prefix reuse |
| 14. What is speculative decoding? | Draft、verification、acceptance rate、額外運算與加速的權衡 |
| 15. Why is decode often memory-bound? | Arithmetic intensity、weight streaming、batch size、Roofline model |
| 16. What changes with MoE inference? | Expert routing、grouped GEMM、expert load balance、All-to-All |

特別注意第 9、10、12、13 題，它們可以串成一條非常自然的追問鏈。

一個你一定要會推導的公式是：

$$M_{\mathrm{KV}} = 2 \times L \times B \times S \times H_{\mathrm{KV}} \times D \times b$$

其中 $L$ 是 layers、$B$ 是 batch size、$S$ 是每個 request 的 sequence length（假設相同）、$H_{\mathrm{KV}}$ 是 KV heads、$D$ 是 head dimension，$b$ 是每個元素的 bytes。這裡先不計 allocator metadata、padding 等額外開銷。

**你要能手算出具體模型的 KV cache 容量，再解釋為什麼 GQA、KV quantization 和 paged allocation 能改善 serving。**

推薦先讀：

- FlashAttention: Fast and Memory-Efficient Exact Attention ↗
- PagedAttention / vLLM（SOSP 2023） ↗
- TensorRT-LLM KV Cache System ↗

**C. TensorRT-LLM Runtime / Serving**

| 可能的面試題目 | 要回答到的重點 |
|---|---|
| 17. Static batching vs continuous batching? | 每個 iteration 動態新增/移除 request、padding 與吞吐量 |
| 18. Design a KV cache manager. | Block pool、mapping、allocation、refcount、eviction、OOM |

| | |
|---|---|
| 19. Explain chunked prefill. | 將長 prompt 拆成 chunks、token budget、TTFT/TPOT 權衡 |
| 20. How do CUDA Graphs help decode? | Launch overhead、capture/replay、dynamic shapes、graph padding |
| 21. Design a high-throughput LLM server. | Request queue → scheduler → model execution → sampling → streaming |
| 22. Why disaggregate prefill and decode? | 資源特性、相互干擾、KV transfer overhead、不同 GPU 配置 |
| 23. What are TTFT, TPOT and throughput? | 衡量方式、p50/p99、不同優化目標可能衝突 |
| 24. TensorRT-LLM vs vLLM vs SGLang? | 執行引擎、kernel、runtime、scheduler、KV management 與部署權衡 |

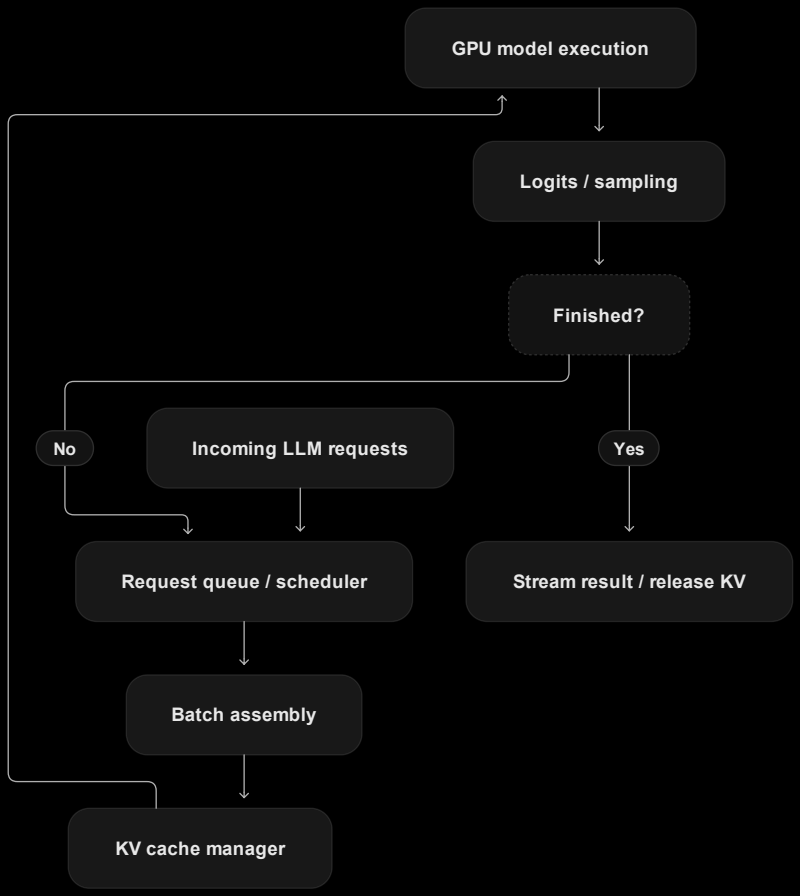
這裡有一個容易準備錯誤的地方：

**2026 年的 TensorRT-LLM，不能只用早期的「TensorRT engine builder + plugins」架構去理解。**

目前官方文件強調 PyTorch-native model authoring、LLM API、PyTorch executor，以及共享的 C++ scheduling、KV cache 和 decoding components。熟悉舊的 TensorRT graph compilation 仍有價值，但不應忽略現代 runtime。

GitHub +2

下面是你應該理解的簡化 request flow：




面試重點不是把架構圖背下來，而是能回答：「如果每個 request 的 context length 不同、GPU memory 不夠、突然有一個 32K token 的 prompt 進來，你的 scheduler 要怎麼決策？」

**D. C++ / Parallel Algorithms / Performance**

| 可能的面試題目 | 要回答到的重點 |
|---|---|
| 25. Serialize / deserialize binary tree. | BFS/DFS、null markers、O(N) time/space；曾出現在公開 NVIDIA inference screen |
| 26. Implement parallel prefix sum (scan). | Work-efficient scan、同步、跨 block 策略 |
| 27. Implement a thread-safe queue. | Mutex、condition variable、shutdown、避免 race / deadlock |
| 28. Explain RAII, move semantics, Rule of Five. | GPU buffers、ownership、lifetime、exception safety |

| | |
|---|---|
| 29. CUDA streams and synchronization? | Async operations、events、dependencies、host/device synchronization |
| 30. How to debug a slow kernel? | Nsight Systems → Nsight Compute → hypothesis → A/B benchmark |
| 31. How do FP16 / BF16 / FP8 / INT4 differ? | Range、precision、scale、accumulation、hardware support |
| 32. How do you prove an optimization works? | Correctness、相同 workload、latency distribution、repeatability、end-to-end gain |

第 30、32 題其實很重要。NVIDIA 自己的 TensorRT-LLM performance docs 也強調以 profiler 識別執行瓶頸，並在測試時控制環境、工作負載與 GPU 設定，而不是只報一個最佳 speedup。

GitHub +2

### 3. 以你目前的背景，最需要補什麼？

你之前做過 Qualcomm LiteRT 相關的 backend / op integration、C++ 實作，也接觸過量化、GPU 計算和 CUDA Graph 等議題。

這對 TensorRT-LLM 是有幫助的，因為你不是從零開始學 inference stack。不過，**做過 inference runtime 跟能獨立最佳化 GPU kernel，仍然是兩個不同層次的能力。**

我的初步評估：

| 領域 | 目前基礎判斷 | 建議 |
|---|---|---|
| C++ software engineering | 有實際專案經驗 | 維持，補 concurrency |
| Inference runtime / delegate | 相對有優勢 | 練習把 LiteRT 經驗對應到 TensorRT-LLM |
| Quantization | 有接觸 | 補上 GPU datatype 與 kernel performance |
| CUDA kernel optimization | 實作與 profiling 深度待確認 | **主要補強** |
| GPU memory hierarchy | 需要系統整理 | **主要補強** |
| Transformer / attention | 需要確認是否能從公式推到 kernel | **主要補強** |
| Serving / KV scheduling | 尚未看到深入的實作案例 | **主要補強** |
| Distributed inference | 尚未確認經驗 | 第二階段再補 |

我覺得你最大的機會是把既有的 runtime / compiler engineering 背景，延伸到 GPU performance optimization。

而不是現在就去補一大堆 distributed training，或背 Blackwell 所有新指令。

**五個特別值得補的知識缺口**

**GPU microarchitecture**

要知道 SM、warp scheduler、register file、shared memory、L1/L2、HBM 之間的關係。進一步能從 register usage 和 shared memory usage 推導 occupancy 限制。

**Performance modeling / profiling**

學 Roofline、arithmetic intensity、effective bandwidth、Nsight Systems 和 Nsight Compute。尤其要會區分 memory bandwidth bound、compute bound、latency bound 與 launch overhead。

**Attention kernel implementation**

不只知道 Transformer 公式，而是可以從 naive attention 推導到 tiling、online softmax、FlashAttention，並解釋 prefill 和 decode kernel 差異。

**Serving runtime internals**

了解 scheduler、batch manager、KV cache manager 的協作，包括 OOM、request cancellation、prefix cache reuse、dynamic batching。

**Triton / CUDA / TensorRT 之間的關係**

理解什麼時候自己寫 kernel、什麼時候用 Triton、何時依靠 cuBLAS / CUTLASS，以及 `torch.compile` 能做什麼、不能做什麼。注意 Triton GPU programming language 與 NVIDIA Triton Inference Server 是兩個不同的產品。

### 4. 我建議的六週準備計畫

先以每週約 6–10 小時設計，不需要一次把所有東西學完。

**Week 1 — CUDA Memory Hierarchy**

Memory coalescing、shared memory、bank conflict、occupancy、register spilling

實作：完成 CUDA matrix transpose：naive → tiled → bank-conflict-free

驗收：能用 profiler 解釋每一版快慢的原因

**Week 2 — CUDA Reduction / Softmax / Triton**

Warp shuffle、parallel reduction、numerical stability、kernel fusion

實作：自己實作 reduction 和 softmax，與 PyTorch / Triton 比較

驗收：能回答不同 shape、block size 下的效能差異

**Week 3 — Transformer / FlashAttention / KV Cache**

Prefill/decode、GQA、attention complexity、online softmax、PagedAttention

實作：用 PyTorch 實作 attention，再研究 tiled attention 的資料流

驗收：能推導 attention 的計算量與記憶體成本

**Week 4 — TensorRT-LLM Runtime / Serving**

LLM API、executor、continuous batching、KV cache manager、CUDA Graph

實作：用小型 LLM 做單 GPU serving 與 concurrency benchmarks

驗收：能解釋 TTFT、TPOT、吞吐量和 scheduler trade-off

**Week 5 — Tensor Core / Quantization / Multi-GPU**

Tiled GEMM、Tensor Core、FP8/FP4、AWQ、TP/PP/EP、NCCL

實作：做 Triton matmul 或 fused RMSNorm，理解 precision/performance

驗收：能以 Roofline 估計優化空間

**Week 6 — Mock Interview / Project Deep Dive**

C++ DSA、CUDA kernel optimization、serving system design、專案追問

實作：完成 2 場模擬面試，整理 LiteRT 經驗和 GPU profiling 案例

驗收：準備可量化、有 before/after 數據的技術故事

**實作時要留下哪些東西？**

如果要把這些練習當作未來面試的材料，每個實驗最好都留下 workload、硬體與環境資訊、correctness test、benchmark 結果，以及 profiler screenshot。

例如 kernel optimization 不只寫「提升 2 倍」，而是要能指出哪個版本、何種 shape、使用多少 registers、是否改善 global load efficiency，以及優化是否引入新的瓶頸。

如果使用你之前提過的 NVIDIA 消費級 GPU，可以從小型模型開始；先確認該版本 TensorRT-LLM 的硬體支援矩陣。CUDA kernel 練習不需要先準備多 GPU 系統。

---

### 5. 精選學習資源：不用全部看完

我把資料分成 CUDA → Triton → TensorRT-LLM 三層，建議按順序閱讀。



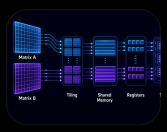
**CUDA 基礎和最佳化**

CUDA C++ Programming Guide ↗

Efficient Matrix Transpose ↗

Nsight Compute Triage Guide ↗

先讀 memory coalescing、shared memory，再做 profiling。



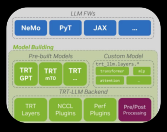
**Triton Kernel Programming**

Fused Softmax Tutorial ↗

Matrix Multiplication Tutorial ↗

兩篇都包含 kernel 實作、資料配置和效能最佳化。

---



**TensorRT-LLM 原始碼與文件**

Repository Architecture / Code Map ↗

Architecture Overview ↗

Performance Analysis ↗

先了解 request flow，再進 scheduler、attention、KV cache。

不需要一開始就研究 Hopper WGMMA、Blackwell TMA、NVFP4 或 Expert Parallelism 的所有細節。這些可以等你已經能自己完成 kernel profiling 與 inference benchmark，再依職缺方向深入。

### 6. 第一階段：從這四題開始

我最建議先測試你的 CUDA 基礎。下面四題很適合用來確認目前的深度。

QUESTION 1

**為什麼 row-major matrix 的 column-wise access 可能造成 uncoalesced global memory access？**

提示

QUESTION 2

**Shared memory tile[32][32] 為何可能有 32-way bank conflict？改成 [32][33] 為何能解決？**

提示

QUESTION 3

**如果一個 kernel 的 occupancy 只有 25%，是否代表它一定很慢？**

提示

QUESTION 4

**你寫了一個 CUDA reduction kernel，比 torch.sum 慢 3 倍，會怎麼 debug？**

提示

**我建議的準備方式是「一個主題 → 理論 → CUDA/C++ 實作 → profiler → 面試追問」。**

這會比每天背 20 題面試題有效得多。尤其你的目標不是一般 ML application engineer，而是 TensorRT-LLM 這種需要真正理解 GPU execution、runtime 和 performance trade-off 的職缺。

接下來最適合從 **CUDA Memory Hierarchy + Coalescing + Shared Memory + Occupancy** 開始，直接配 matrix transpose 實驗。這一組打穩之後，reduction、softmax、FlashAttention 和 TensorRT-LLM kernel 都比較容易接上。

**你說：**

再幫我想一下, 像是 C++ 的特性複習, cuda 的 programming 那些感覺也要順便複習幫我整理一個 map, 我可以在 tensorrt-llm repo 裡面給 codex 做的任務帶我去看

處理時間為 3m 34s ›

---

**ChatGPT 說：**

## TensorRT-LLM 面試準備：C++ × CUDA × Runtime 學習地圖

我建議不要單純列一份 C++ / CUDA 的複習清單，而是把 **TensorRT-LLM repo 當成教材，讓 Codex 帶你讀 code、做實驗、接受技術追問**。

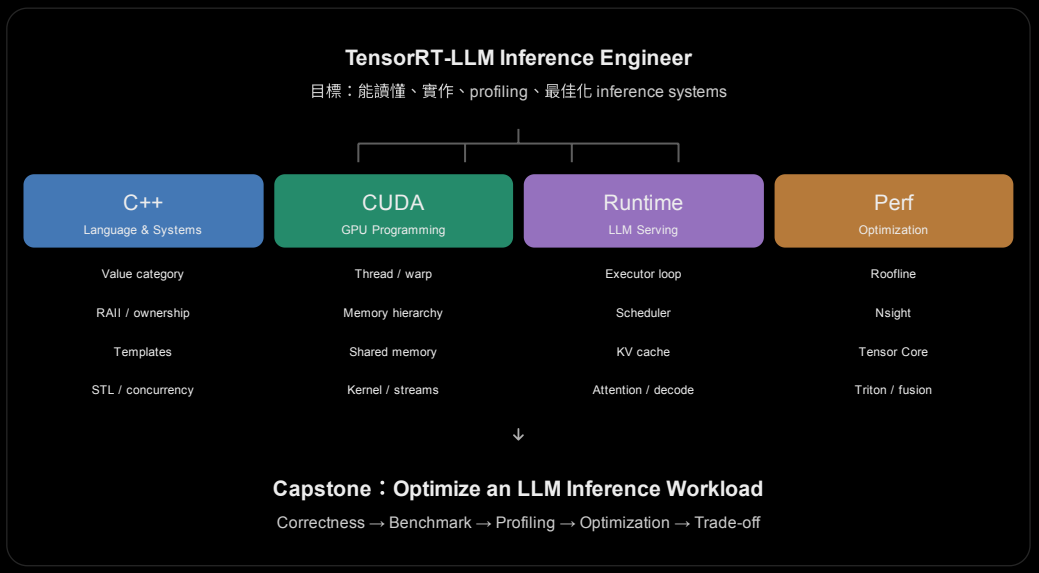
這樣可以同時達成三個目標：

1. 複習 C++17/20、CUDA Programming、GPU Architecture。
2. 搞懂 TensorRT-LLM 真正的程式架構和最佳化策略。
3. 累積面試時可以講的實作案例，不只是背理論。

我查了目前的 NVIDIA/TensorRT-LLM ↗ repository。現在應該以 **PyTorch executor + C++ runtime components + CUDA/Triton kernels** 為主要閱讀路線，而不是從舊版 TensorRT engine backend 開始。

GitHub +1

## 1. 完整 Knowledge Map




### 你需要複習的細項

| 模組 | 必修內容 | 延伸內容 |
|---|---|---|
| C++ Language | `const`、`&` / `&&`、lvalue/rvalue、move semantics、RAII、smart pointers | Perfect forwarding、RVO/NRVO、type traits |
| C++ Templates | Function/class template、specialization、`constexpr`、`if constexpr` | Variadic templates、concepts、SFINAE |
| C++ Systems | Memory layout、alignment、lifetime、STL、threads、mutex、atomic | Lock-free、memory ordering、cache-line false sharing |
| CUDA Basics | Grid/block/thread、kernel launch、device memory、streams | CUDA events、graphs、pinned memory、async allocation |
| CUDA Architecture | SM/warp、coalescing、register、L1/L2、shared memory | Bank conflicts、occupancy、spilling、warp divergence |
| CUDA Algorithms | Reduction、scan、transpose、softmax、tiled GEMM | Warp shuffle、Tensor Core、persistent kernel |
| LLM Execution | Transformer、prefill/decode、GQA、KV cache | FlashAttention、paged KV、speculative decoding |
| Runtime | Scheduler、continuous batching、request lifecycle | Chunked prefill、overlap scheduler、disaggregated serving |
| Performance | Roofline、latency、bandwidth、Nsight Systems/Compute | Tensor Core utilization、Triton、kernel fusion |

**閱讀順序：C++ ownership → CUDA memory → CUDA kernels → C++ runtime → scheduler/KV cache → profiling → advanced kernels。**

你以前複習的 Rule of Five、perfect forwarding、RVO、`this auto&&`、templates，這次可以直接拿 TensorRT-LLM 的實際程式碼複習，會更容易知道為什麼這些語言特性重要。

## 2. Repo Reading Map：從哪裡開始？

下面是我實際查到的 repo 路徑。Codex 執行時仍應先驗證當前 checkout，因為 TensorRT-LLM 的 main branch 變動很頻繁。

SYSTEM ENTRY POINT　先看這裡

**llmapi/llm.py** ↗　Python

High-level `LLM` API；追蹤 `generate()` 如何進入 executor。

**_torch/pyexecutor/py_executor.py** ↗　Python

PyExecutor event loop、model forward、sampling 和 overlap。

**pyexecutor/scheduler/scheduler.py** ↗　Python / C++ binding

Request 排程、token budget、chunked prefill、capacity scheduling。

↓ C++ SYSTEMS

**cpp/include/.../runtime/bufferManager.h** ↗

C++ ownership、`unique_ptr`、`shared_ptr`、GPU allocation、memory lifetime。

**cpp/include/.../executor/executor.h** ↗

`std::optional`、configuration objects、request / executor API。

**cpp/.../batch_manager/kvCacheManager.cpp** ↗

KV blocks、allocation、reuse、resource management。

↓ GPU KERNELS

**kernels/samplingTopPKernels.cu** ↗

CUDA indexing、CUB reduction/scan、shared memory、template kernels。

**kernels/fusedLayernormKernels/ws_layernorm.cuh** ↗

高階 RMSNorm / LayerNorm、warp specialization、register tiling、shared memory。

這些檔案分別能對應到 C++ systems、CUDA programming 與 serving internals，而不會一開始就掉進幾千行的 FlashAttention kernel。

GitHub +6

### 3. C++ 複習地圖：直接從 repo 學語言特性

我會先集中在五組，這些是閱讀 C++ inference runtime 最容易遇到、也最可能被追問的。

| 主題 | 要能回答的問題 | 對應 repo |
|---|---|---|
| Object Model / Value Category | `const T&`、`T&&`、`std::move`、RVO/NRVO、何時會 copy？ | `executor.h` |
| RAII / Ownership | `unique_ptr` vs `shared_ptr`、custom deleter、GPU buffer 何時釋放？ | `bufferManager.h` |
| Templates / Metaprogramming | `typename`、`constexpr`、`if constexpr`、type traits、template specialization | `ws_layernorm.cuh` |
| Memory / Concurrency | `std::thread`、mutex、atomic、condition variable、data race | `batch_manager/` |
| Systems Interface | alignment、pointer lifetime、host/device code、`noexcept`、CMake | `runtime/`、`kernels/` |

有幾個細節建議特別複習：

- `std::move` 本身不移動資料，只是轉換 value category；後續操作是否呼叫 move constructor 才是關鍵。
- C++17 的 prvalue 在特定情況有 guaranteed copy elision，但 NRVO 不保證一定發生。

- `T&&` 在型別推導情境可能是 forwarding reference，但並非所有 `T&&` 都是。
- `this auto&& self` 這種 explicit object parameter 遞迴寫法屬於 **C++23**，不要誤以為是 C++17 特性。

**最好的 C++ 練習：GPU Buffer 的 Ownership**

你可以從 `BufferManager` 開始。它有 `IBuffer::UniquePtr`、`ITensor::UniquePtr`、`std::shared_ptr<CudaStream>`，而且涉及同步與非同步 GPU memory allocation。

GitHub

Codex 要帶你解釋的不只是每個 class，而是這些設計背後的問題：

> 如果 GPU allocation 使用 `cudaMallocAsync`，C++ object 的 destructor 被呼叫後，記憶體何時才真正能安全地重用？

這個問題能把 RAII、stream ordering、asynchronous execution 和 lifetime 全部串起來。

## 4. CUDA Programming Map：從小 kernel 一路做到 LLM

| Level | 你要自己實作的東西 | 核心觀念 |
|---|---|---|
| CUDA 0 | Vector Add | Thread indexing、grid-stride loop、bounds check |
| CUDA 1 | Matrix Transpose | Coalescing、shared memory、bank conflicts |
| CUDA 2 | Sum / Max Reduction | Warp shuffle、block reduction、同步 |
| CUDA 3 | Prefix Sum / Scan | Work-efficient parallel algorithms、CUB |
| CUDA 4 | Softmax | Numerical stability、online reduction、memory traffic |
| CUDA 5 | RMSNorm | Kernel fusion、vectorization、register pressure |
| CUDA 6 | Tiled GEMM | Tiling、data reuse、shared memory、Tensor Core |
| CUDA 7 | Attention | $QK^T$、softmax、PV、FlashAttention |
| CUDA 8 | Kernel Profiling | Nsight、Roofline、occupancy、bottleneck analysis |

其中前四個應該用 CUDA C++ 自己寫，不要一開始就交給 Triton。到 RMSNorm、GEMM 再比較 PyTorch、CUDA 和 Triton。

TensorRT-LLM 的 `samplingTopPKernels.cu` 包含 CUB `BlockReduce`、scan callback、kernel launch、stream 等具體案例，所以可以在寫完 reduction 後回去對照 production code。

GitHub

## 5. Codex 學習任務清單

我會把整個流程拆成 10 個任務。每個任務都要有 **Code Reading**、**Exercise**、**Validation**、**Interview Questions** 四個輸出。

**學習進度** 0/10

可勾選已完成的任務

All | P0 | C++ | CUDA | Runtime | Kernel | Perf | Systems

M00 Repo Architecture Walkthrough

llmapi/llm.py → py_executor.py

M01 Modern C++ / RAII

runtime/bufferManager.h

分析 GPU buffers、ownership 和 move semantics

驗收：能設計一個 move-only GPU buffer wrapper

複製 Codex 任務

- M02 C++ Templates & Const
  executor/executor.h + ws_layernorm.cuh
- M03 C++ Concurrency
  batch_manager/ + executor/
- M04 CUDA Memory & Transpose
  kernels/ + standalone lab
- M05 Reduction / Scan / Sampling
  kernels/samplingTopPKernels.cu
- M06 CUDA Streams & Graphs
  _torch/compilation/ + py_executor.py
- M07 Scheduler & KV Cache
  scheduler.py + kvCacheManager.cpp
- M08 RMSNorm / Triton Optimization
  fusedLayernormKernels/ + triton_kernels/
- M09 Nsight / Performance Case Study
  perf-analysis.md + trtllm-bench

這個清單的順序不必完全照編號。我的建議是先做 `M01 → M02 → M04 → M05 → M00 → M07`，你會同時建立語言、GPU 和系統層面的連結。

---

### 6. 給 Codex 的總教練 Prompt

這段放到 TensorRT-LLM repo 的 Codex session 開頭。目的是讓 Codex 用 **code reading + teaching + interview** 模式，而不是自動幫你把所有程式寫完。

**TensorRT-LLM Interview Mentor** 複製

純文字

```
Role: Senior C++ / CUDA / LLM inference mentor.

Goal: Prepare me for a TensorRT-LLM engineering interview through real source code.

Rules:
- Inspect the actual repository first.
- Explain real code with file:line references.
- One concept and one question at a time.
- Let me attempt code before showing solutions.
- Use isolated labs, not production modifications.
- Validate correctness before profiling.
- Explain every optimization trade-off.
- End each lesson with interview follow-ups.

Start with:
C++ RAII and GPU buffer ownership.
Read bufferManager.h and related definitions.
```

### 7. 三個我最推薦優先做的實戰任務

### Task A — C++：設計一個 GPU Buffer Class

M01 · C++17/20 · RAII / Rule of Five / Move Semantics

讓 Codex 先帶你讀 `BufferManager`，接著設計一個簡化的 GPU buffer：

```cpp
class DeviceBuffer {
public:
    explicit DeviceBuffer(size_t bytes);
    ~DeviceBuffer();

    DeviceBuffer(const DeviceBuffer&) = delete;
    DeviceBuffer& operator=(const DeviceBuffer&) = delete;

    DeviceBuffer(DeviceBuffer&& other) noexcept;
    DeviceBuffer& operator=(DeviceBuffer&& other) noexcept;

private:
    void* ptr_ = nullptr;
    size_t bytes_ = 0;
};
```

任務不是直接寫出答案，而是先回答：

- 為什麼 delete copy constructor？
- Move constructor 如何轉移 ownership？
- Move assignment 怎麼避免 memory leak？
- Destructor 是否需要 `noexcept`？
- 如果 buffer 和 CUDA stream 的生命週期互相依賴，怎麼處理？

再讓 Codex 要求你自行寫出 `.cpp`，並補上分配失敗、move self-assignment、double free 等測試。

### Task B — CUDA：自己最佳化 Matrix Transpose

M04 · CUDA C++ · Memory Coalescing / Shared Memory

這題非常適合模擬面試官給你一個 naive kernel，要求逐步優化。

你的實驗版本

| V1 Naive | V2 Tiled | V3 Padded |
| --- | --- | --- |
| Direct transpose | Shared memory | Avoid bank conflict |

比較矩陣大小 256×256、1024×1024、2048×2048，以及非 32 倍數的尺寸，記錄 latency 和 effective bandwidth。

再讓 Codex 反問你：「如果 tiled transpose 已經達到接近 memcpy 的 bandwidth，還有必要提高 occupancy 嗎？」

這種問法比單純問 occupancy 定義更接近性能最佳化面試。

### Task C — Runtime：手動追 Scheduler 與 KV Cache

M07 · TensorRT-LLM · Scheduler / KV Block Manager

閱讀：

- `tensorrt_llm/_torch/pyexecutor/scheduler/scheduler.py`
- `cpp/tensorrt_llm/batch_manager/kvCacheManager.cpp`
- Scheduler 文件 ↗

目前 scheduler 文件區分 `CapacityScheduler`（資源可否配置）與 `MicroBatchScheduler`（本 iteration 讓哪些 request 執行）。

GitHub

我會要求 Codex 建立一個簡化模擬，例如三個 requests：

| Request | Prompt tokens | 要生成的 tokens |
| --- | --- | --- |

| | | |
|---|---|---|
| A | 128 | 64 |
| B | 2,048 | 128 |
| C | 512 | 32 |

假設每個 KV block 存 16 tokens、GPU 有固定 block budget。讓你逐輪追蹤 prefill、decode、KV allocation、request admission 與完成後釋放。這是教學用的簡化模型，不等於 TensorRT-LLM 真正的完整排程策略。

等你能自己跑完，再讀 production implementation，效果會比直接打開 `kvCacheManager.cpp` 從頭看好很多。

---

### 8. 最後要把學習成果轉成面試能力

每完成一個任務，讓 Codex 要求你交一份很短的結果：

| 證明什麼能力 | 你應該留下的產出 |
|---|---|
| C++ correctness | 編譯通過的程式、unit tests、ownership 說明 |
| CUDA correctness | CPU/PyTorch reference、edge-case tests |
| CUDA performance | latency、effective bandwidth、kernel 資源使用量 |
| Systems understanding | Call graph、資料結構圖、request lifecycle |
| Optimization reasoning | Bottleneck hypothesis、修改原因、before/after |
| Interview communication | 2 分鐘技術說明 + 追問回答 |

做效能實驗時可以依照 TensorRT-LLM 官方 Performance Analysis Guide ↗ 操作。官方文件有 `nsys`、NVTX、特定 iterations profiling 等方式可供參考。

GitHub

### 我建議你現在的起點

**先完成 M01（C++ RAII）和 M04（CUDA Memory/Transpose），再進入 M05（Reduction）與 M07（KV Cache）。**

C++ 與 CUDA 交錯練習會比把 C++ 全部讀完、接著再開始 CUDA 更容易建立連結。

而且這四個任務可以產生四個很直接的面試題材：資源生命週期、記憶體最佳化、平行演算法，以及 inference memory management。等到這些能夠自己解釋和實作，再去讀 Tensor Core、FlashAttention、Triton 和 Multi-GPU，會比較扎實。